Lecture 12
Augmenting Data Structures

Overview

- It's unusual to have to design an all-new data structure from scratch
- It's more common to take a data structure that you know and store additional information in it
- With the new information, the data structure can support new operations
- Challenge
  - How to correctly maintain the new information without loss of efficiency

Dynamic Order Statistics

- We want to support the usual dynamic-set operations from Red-Black trees, plus
  - $\text{OS} - \text{SELECT}(x, i)$: return pointer to node containing the i th smallest key of the subtree rooted at x
  - $\text{OS} - \text{RANK}(T, x)$: return the rank of x in the linear order determined by an inorder walk of T

Augment R-B Tree

- Augment by storing in each node x
  - $x.size =$ # of nodes in subtree rooted at x
  - Includes x itself
  - Does not include leaves (NILs)

- It follows that

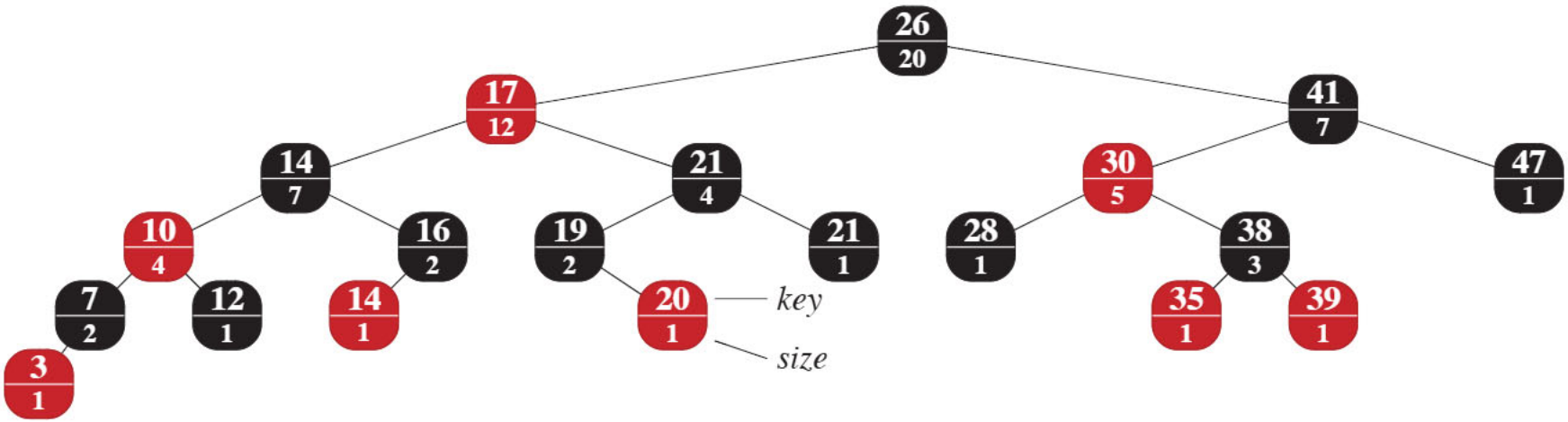
$$x.size = x.left.size + x.right.size + 1$$

OS-SELECT

```
OS-SELECT(x, i)
r = x.left.size + 1    // rank of x within the subtree rooted at x
if i == r
    return x
elseif i < r
    return OS-SELECT(x.left, i)
else return OS-SELECT(x.right, i − r)
```

Example

- Try $\text{OS} - \text{SELECT}(T.root, 19)$



Analysis

- Correctness

r = rank of x within subtree rooted at x.

- If $i = r$, then we want x.
- If $i < r$, then ith smallest element is in x's left subtree, and we want the ith smallest element in the subtree.
- If $i > r$, then ith smallest element is in x's right subtree, but subtract off the r elements in x's subtree that precede those in x's right subtree.
- Like the randomized SELECT algorithm.

- Running time: $O(\lg n)$

OS-RANK

```
OS-RANK(T, x)
r = x.left.size + 1                    // rank of x within the subtree rooted at x
y = x                                  // root of subtree being examined
while y ≠ T.root
    if y == y.p.right                  // if root of a right subtree ...
        r = r + y.p.left.size + 1      // ... add in parent and its left subtree
    y = y.p                            // move y toward the root
return r
```

- Why it works?

Loop invariant: At start of each iteration of **while** loop, r = rank of $x.key$ in subtree rooted at y.

- Running time: $O(\lg n)$

OS-RANK

```
OS-RANK(T, x)
r = x.left.size + 1                    // rank of x within the subtree rooted at x
y = x                                  // root of subtree being examined
while y ≠ T.root
    if y == y.p.right                  // if root of a right subtree ...
        r = r + y.p.left.size + 1      // ... add in parent and its left subtree
    y = y.p                            // move y toward the root
return r
```

- Why it works?

- Running time: $O(\lg n)$

Maintaining Subtree Sizes

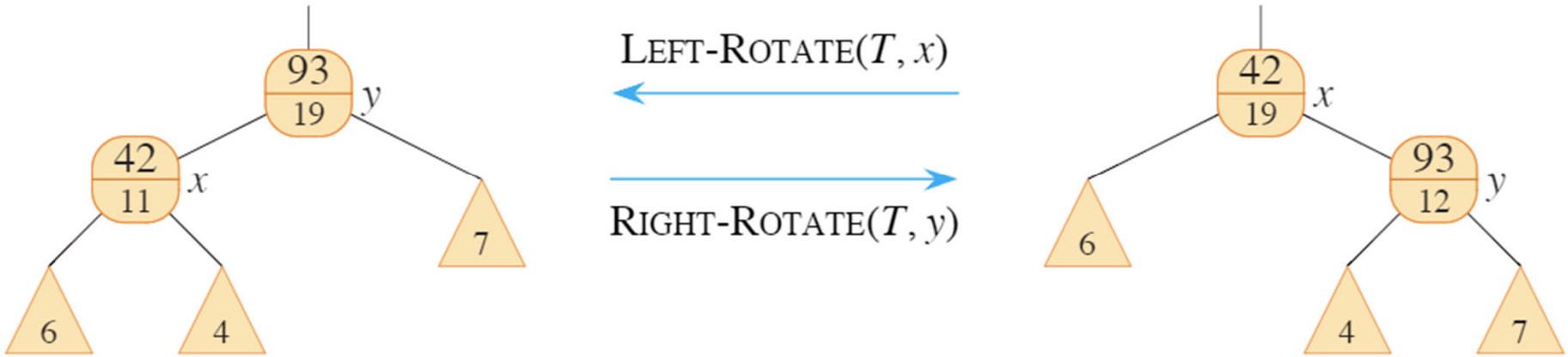
- Need to maintain *size* attributes during insert and delete operations
- Need to maintain them efficiently, i.e., in $O(\lg n)$ time

Insertion

- During pass downward, we know that the new node will be a descendant of each node we visit, and only of these nodes. Therefore, increment *size* attribute of each node visited
- Then there's the fix-up pass
  - Goes up the tree
  - Changes colors $O(\lg n)$ times
  - Performs at most two rotations
- Color changes don't affect subtree sizes.
- Rotations do!

Insertion

- Determine new sizes based on old sizes and sizes of children



$$y.size = x.size$$

$$x.size = x.left.size + x.right.size + 1$$

Insertion

- Similar for right rotation
- Can update in $O(1)$ time per rotation. Therefore $O(1)$ time spent updating *size* attributes during fix-up
- Therefore, $O(\lg n)$ to insert

Delete

- Constant work at each level + at most 3 rotations
- Therefore, $O(\lg n)$ to delete

General Methodology

- Step 1: choose an underlying data structure
- Step 2: determine additional information to maintain.
- Step 3: verify that we can maintain additional information for existing data structure operations.
- Step 4: develop new operations

- Don’t need to do these steps in strict order!

When to Use Red-black Tree?

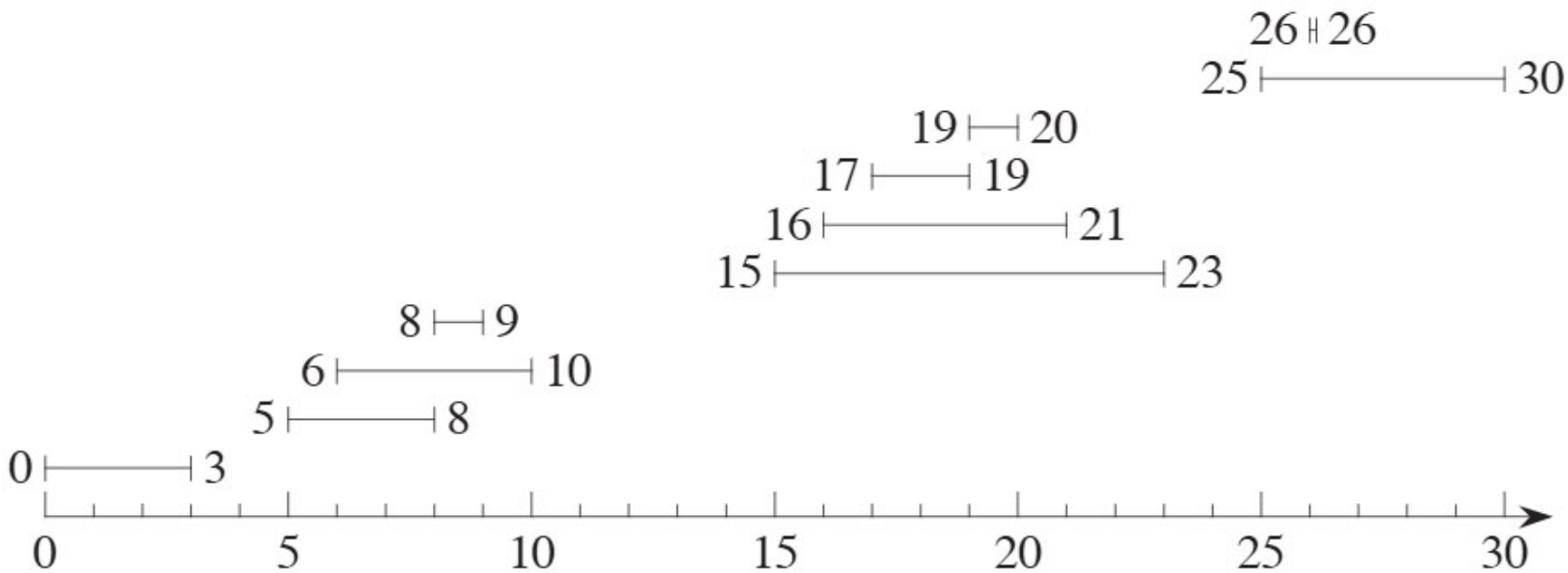
- Red-black trees are particularly amenable to augmentation

Theorem

Augment a R-B tree with attribute f, where $x.f$ depends only on information in x, $x.left$, and $x.right$ (including $x.left.f$ and $x.right.f$). Then can maintain values of f in all nodes during insert and delete without affecting $O(\lg n)$ performance.

Interval Tree

- Maintain a set of intervals, e.g., time intervals



Operations

- Interval i has two attributes: $i.low$ and $i.high$
- Operations
  - Interval $-$ Insert(T, x): insert a new interval x (pointer)
  - Interval $-$ Delete(T, x): delete an interval x
  - Interval $-$ Search(T, i): return pointer to a node x in T such that $x.int$ overlaps with interval i
    - Any overlapping node in T is OK
    - Return pointer to sentinel NIL if no overlapping node in T

When Do Two Intervals Overlap?

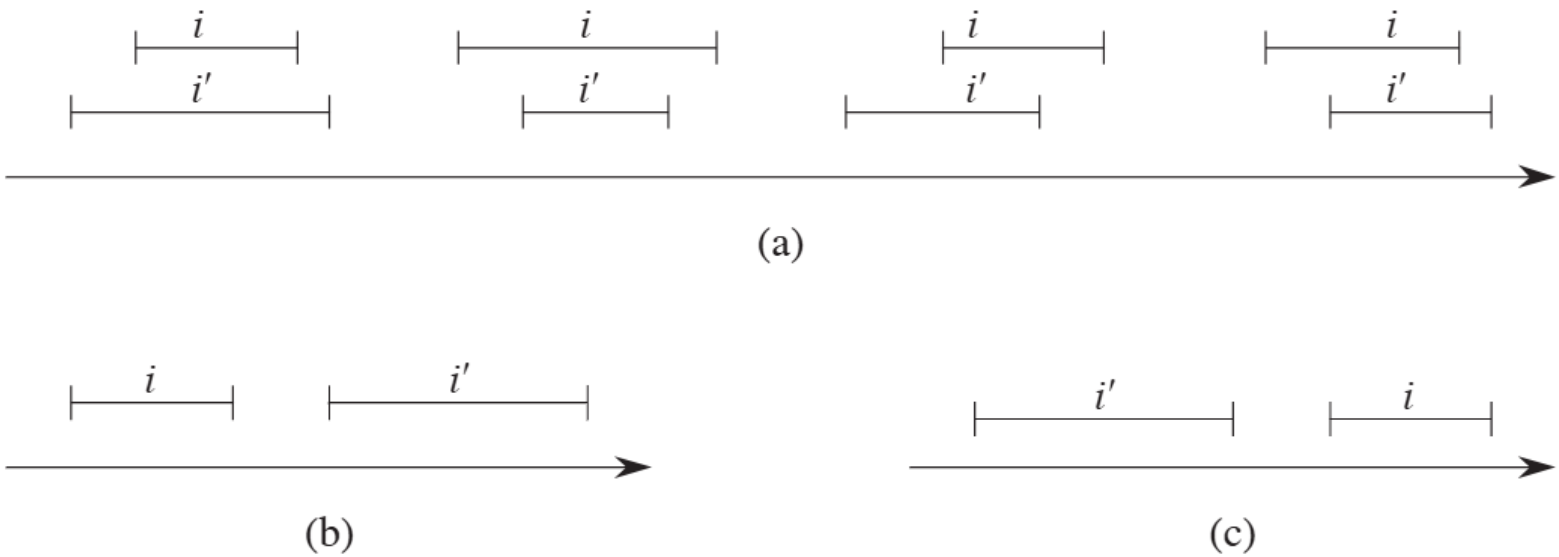


Figure 17.3 The interval trichotomy for two closed intervals i and i'. **(a)** If i and i' overlap, there are four situations, and in each, $i.low \leq i'.high$ and $i'.low \leq i.high$. **(b)** The intervals do not overlap, and $i.high < i'.low$. **(c)** The intervals do not overlap, and $i'.high < i.low$.

When Do Two Intervals Overlap?

- Intervals i and j overlap if and only if

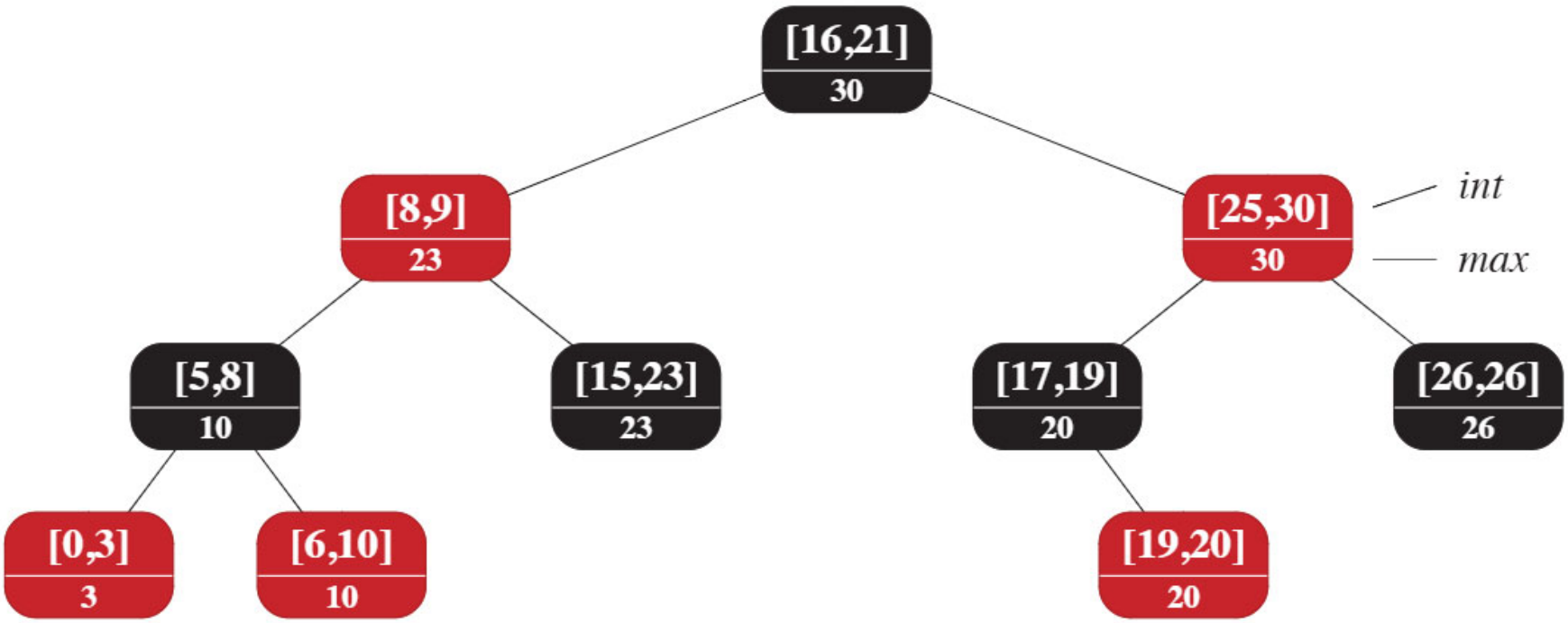
$$i.low \leq j.high \text{ and } j.low \leq i.high$$

- Four cases
  - Case 1: $i.low \leq j.high$ and $i.high \leq j.low$ (no overlap)
  - Case 2: $i.low \leq j.high$ and $i.high \geq j.low$ (overlap)
  - Case 3: $i.low \geq j.high$ and $i.high \leq j.low$ (impossible)
  - Case 4: $i.low \geq j.high$ and $i.high \geq j.low$ (no overlap)

Interval Trees

- Use red black trees
  - Each node x contains interval $x.int$
  - Key is low endpoint $x.int.low$
  - Inorder walk would list intervals sorted by low endpoint
- Each node x additionally contains
  - $x.max$: max endpoint value in subtree rooted at x

Example


[16,21]
30
[8,9]
23
[25,30]
30
int
max
[5,8]
10
[15,23]
23
[17,19]
20
[26,26]
26
[0,3]
3
[6,10]
10
[19,20]
20

How to Maintain MAX Information?

$$x.max = \max(x.int.high, x.left.max, x.right.max)$$

- Note it is possible that $x.left.max > x.right.max$

Maintaining the MAX Information

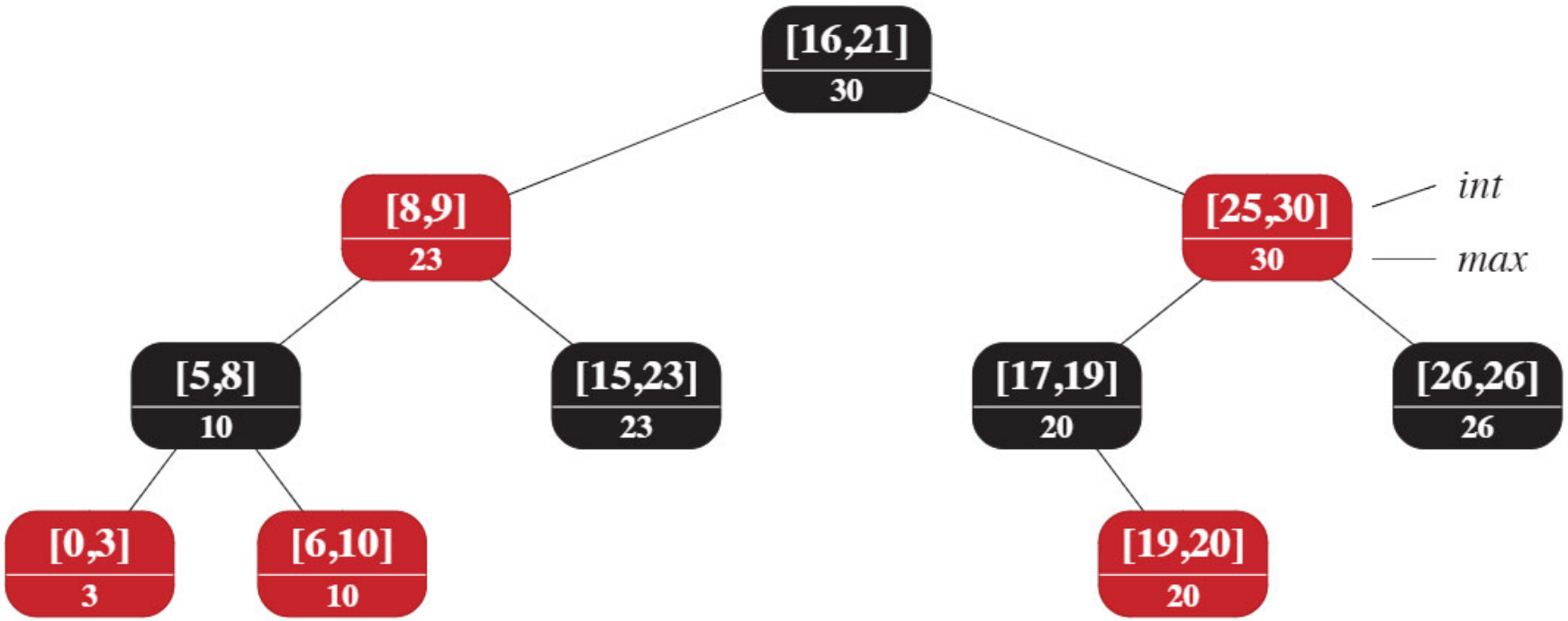
- This is easy— $x.max$ depends only on
  - Information in x : $x.int.high$
  - Information in $x.left$: $x.left.max$
  - Information in $x.right$: $x.right.max$

- Previous theorem tells us that Interval tree can be maintained in $O(\lg n)$ time

Developing New Operations

```
INTERVAL-SEARCH(T, i)
x = T.root
while x ≠ T.nil and i does not overlap x.int
    if x.left ≠ T.nil and x.left.max ≥ i.low
        x = x.left    // overlap in left subtree or no overlap in right subtree
    else x = x.right  // no overlap in left subtree
return x
```

Example



- Example
  - Search for [14,16] and [12,14]

Intuition: Two Cases of Going Left

Case 1

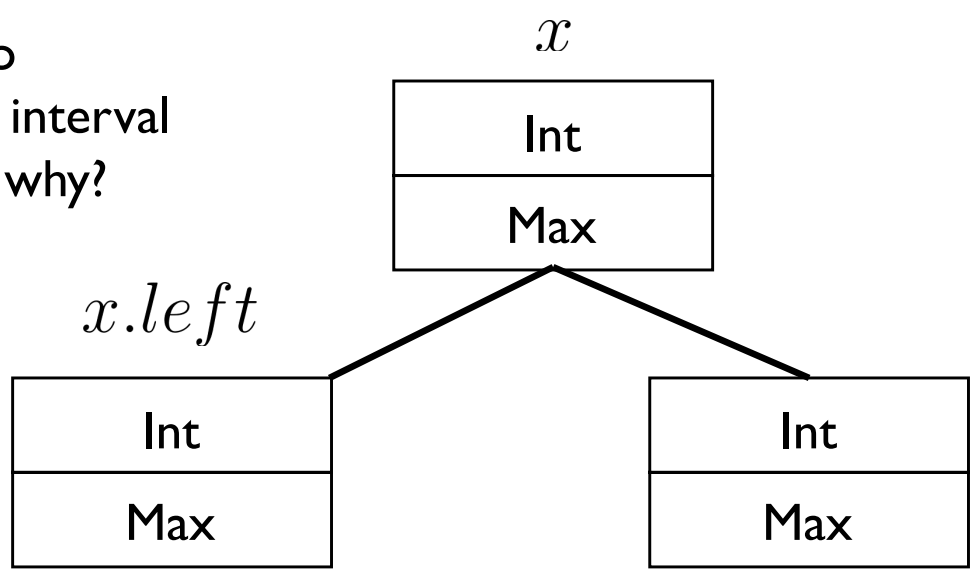
i

Impossible to overlap with interval on the right, why?

Case 2

i

Must overlap with at least one interval on the left, why?




```
while x ≠ T.nil and i does not overlap x.int
    if x.left ≠ T.nil and x.left.max ≥ i.low
        x = x.left    // overlap in left subtree or no overlap in right subtree
```

Correctness

Theorem

If search goes right, then either:

- There is an overlap in right subtree, or
- There is no overlap in either subtree.

If search goes left, then either:

- There is an overlap in left subtree, or
- There is no overlap in either subtree.

Reading Assignment

- Chapter 17